

PEMBELAJARAN MESIN

INTRODUCTION TO NEURAL NETWORKS DENGAN SCIKIT-LEARN

Dr. Arya Adhyaksa Waskita
aawaskita@gmail.com

8 April 2026



REVIEW: TRADITIONAL ML VS NEURAL NETWORKS

Traditional Machine Learning

- Linear models, Decision Trees, SVM
- Feature engineering critical
- Interpretable models
- Good untuk structured data
- Less computational requirements

```
1  # Contoh traditional ML
2  from sklearn.ensemble import RandomForestClassifier
3  model = RandomForestClassifier()
4  model.fit(X_train, y_train)
```

REVIEW: TRADITIONAL ML VS NEURAL NETWORKS

Neural Networks

- Multi-layer perceptrons, Deep Learning
- Automatic feature learning
- "Black box" models
- Excellent untuk unstructured data
- High computational requirements

```
1 # Contoh neural network
2 from sklearn.neural_network import MLPClassifier
3 model = MLPClassifier(hidden_layer_sizes=(100,))
4 model.fit(X_train, y_train)
```

REVIEW: TRADITIONAL ML VS NEURAL NETWORKS

Mengapa Neural Networks Penting?

- State-of-the-art performance di banyak domains
- Handle complex non-linear relationships
- Scalable untuk very large datasets
- Foundation untuk Deep Learning

AGENDA

KONSEP NEURAL NETWORKS

IMPLEMENTASI DENGAN SCIKIT-LEARN

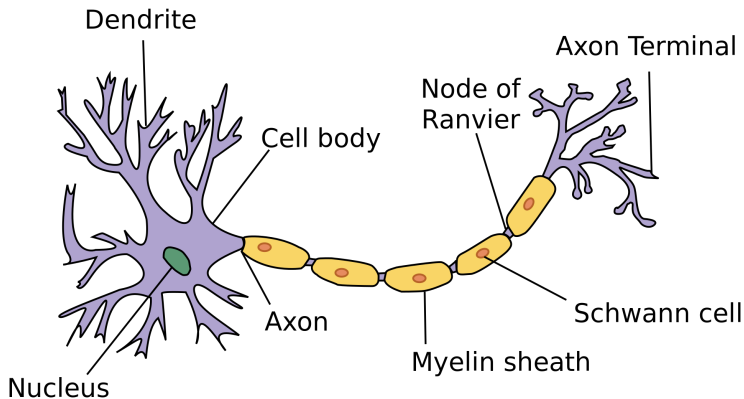
MLP UNTUK REGRESSION

LIVE CODING

PENUTUP

BIOLOGICAL INSPIRATION: OTAK MANUSIA

Neural networks terinspirasi dari biological neurons¹²



¹<https://towardsdatascience.com/>

[explain-like-im-five-artificial-neurons-b7c475b56189/](https://towardsdatascience.com/explain-like-im-five-artificial-neurons-b7c475b56189/)

²<https://www.nature.com/articles/s41467-025-60078-9>

BIOLOGICAL INSPIRATION: OTAK MANUSIA

Biological Neuron:

- **Dendrites:** Receive signals
- **Cell Body:** Process signals
- **Axon:** Transmit signals
- **Synapses:** Connections between neurons

Artificial Neuron (Perceptron):

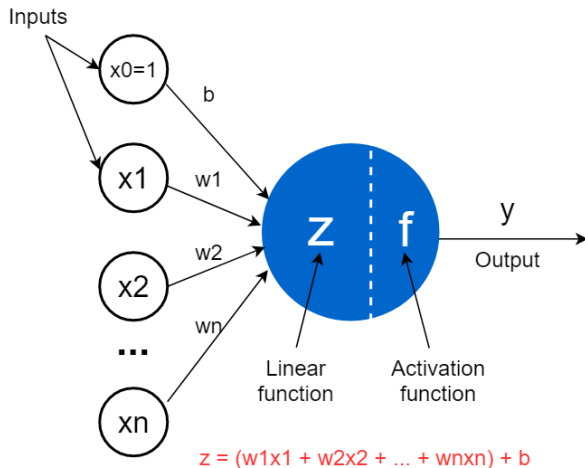
- **Inputs:** Features/activations
- **Weights:** Connection strengths
- **Activation Function:** Decision function
- **Output:** Prediction/activation

BIOLOGICAL INSPIRATION: OTAK MANUSIA

```
1  # Single neuron (perceptron) operation
2  def perceptron(inputs, weights, bias):
3      weighted_sum = np.dot(inputs, weights) + bias
4      output = activation_function(weighted_sum)
5      return output
```


ARTIFICIAL NEURON (PERCEPTRON)

Mathematical model dari biological neuron³



$$y = f(z)$$

³<https://towardsdatascience.com/>

ARTIFICIAL NEURON (PERCEPTRON)

Perceptron Operation:

1. Hitung weighted sum:

$$z = \sum_{i=1}^n w_i x_i + b$$

2. Apply activation function:

$$a = f(z)$$

Components:

- x_i : Input features
- w_i : Weights (parameters)
- b : Bias term
- f : Activation function
- a : Output activation

ARTIFICIAL NEURON (PERCEPTRON)

```
1 import numpy as np
2
3 def perceptron_forward(X, W, b, activation):
4     z = np.dot(X, W) + b    # Weighted sum
5     a = activation(z)        # Activation function
6     return a
```

ACTIVATION FUNCTIONS

Non-linear functions yang menentukan neuron output

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Common activation functions
5 def sigmoid(x):
6     return 1 / (1 + np.exp(-x))
7
8 def relu(x):
9     return np.maximum(0, x)
10
11 def tanh(x):
12     return np.tanh(x)
13
14 def softmax(x):
15     exp_x = np.exp(x - np.max(x)) # Numerical
16     stability
17     return exp_x / np.sum(exp_x)
```

ACTIVATION FUNCTIONS

Non-linear functions yang menentukan neuron output

```
18 # Plot activation functions
19 x = np.linspace(-5, 5, 100)
20 plt.figure(figsize=(12, 8))
21
22 plt.subplot(2, 2, 1)
23 plt.plot(x, sigmoid(x))
24 plt.title('Sigmoid')
25 plt.grid(True)
26
27 plt.subplot(2, 2, 2)
28 plt.plot(x, relu(x))
29 plt.title('ReLU')
30 plt.grid(True)
31
32 plt.subplot(2, 2, 3)
33 plt.plot(x, tanh(x))
34 plt.title('Tanh')
35 plt.grid(True)
```

ACTIVATION FUNCTIONS

Non-linear functions yang menentukan neuron output

```
37 plt.subplot(2, 2, 4)
38 x_softmax = np.array([1.0, 2.0, 3.0])
39 plt.bar(range(3), softmax(x_softmax))
40 plt.title('Softmax (Output)')
41 plt.grid(True)
42
43 plt.tight_layout()
44 plt.show()
```

ACTIVATION FUNCTIONS

Activation Functions Comparison:

SIGMOID

$$f(z) = \frac{1}{1 + e^{-z}}$$

Range: (0, 1)

Use: Output layer untuk binary classification

RELU (RECTIFIED LINEAR UNIT)

$$f(z) = \max(0, z)$$

Range: $[0, \infty)$

Use: Hidden layers (most common)

ACTIVATION FUNCTIONS

Activation Functions Comparison:

TANH

$$f(z) = \tanh(z)$$

Range: $(-1, 1)$

Use: Hidden layers

SOFTMAX

$$f(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Use: Output layer untuk multi-class classification

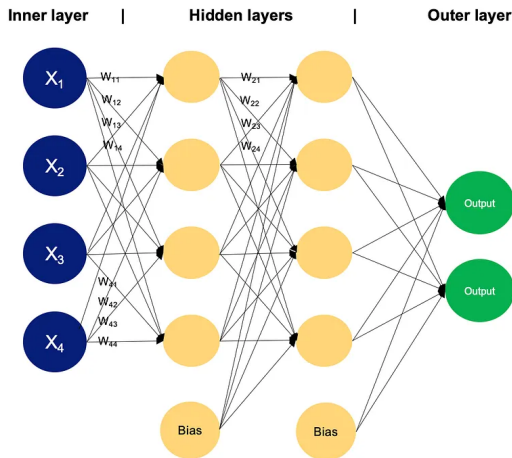
ACTIVATION FUNCTIONS

Why Non-linear?

- Enable learning complex patterns
- Without non-linearity, NN = linear model
- Multiple layers + non-linearity = universal function approximators

MULTI-LAYER PERCEPTRON (MLP) ARCHITECTURE

Network dengan multiple layers of neurons⁴



⁴<https://medium.com/@nlunge786/>

MULTI-LAYER PERCEPTRON (MLP) ARCHITECTURE

Network dengan multiple layers of neurons MLP

Components:

- **Input Layer:** Receives features
- **Hidden Layers:** Process information
- **Output Layer:** Produces predictions
- **Connections:** Weighted links between neurons

Forward Propagation:

1. Input $\rightarrow$ Hidden Layer 1
2. Hidden Layer 1 $\rightarrow$ Hidden Layer 2
3. ...
4. Hidden Layer N $\rightarrow$ Output Layer

Mathematical Representation:

$$a^{[l]} = f^{[l]}(W^{[l]}a^{[l-1]} + b^{[l]})$$

MULTI-LAYER PERCEPTRON (MLP) ARCHITECTURE

Network dengan multiple layers of neurons

```
1  # MLP dengan 2 hidden layers
2  mlp = MLPClassifier(
3      hidden_layer_sizes=(100, 50), # 100 neurons di layer 1, 50 di layer 2
4      activation='relu',
5      solver='adam'
6  )
```

AGENDA

KONSEP NEURAL NETWORKS

IMPLEMENTASI DENGAN SCIKIT-LEARN

MLP UNTUK REGRESSION

LIVE CODING

PENUTUP

DATASET: MNIST HANDWRITTEN DIGITS

```
1 from sklearn.datasets import fetch_openml
2 from sklearn.model_selection import train_test_split
3 from sklearn.preprocessing import StandardScaler
4 import matplotlib.pyplot as plt
5 import numpy as np
6
7 # Load MNIST dataset (handwritten digits)
8 print("Loading MNIST dataset...")
9 X, y = fetch_openml('mnist_784', version=1, return_X_y
10                      =True, as_frame=False)
11
12 # Convert to proper data types
13 X = X.astype(np.float32)
14 y = y.astype(np.int32)
15
16 print(f"Dataset shape: {X.shape}")
17 print(f"Labels: {np.unique(y)}")
```

DATASET: MNIST HANDWRITTEN DIGITS

```
18  # Untuk demonstrasi, kita gunakan subset (  
    computational efficiency)  
19  sample_size = 10000  
20  indices = np.random.choice(len(X), sample_size,  
    replace=False)  
21  X = X[indices]  
22  y = y[indices]  
23  
24  print(f"Subset shape: {X.shape}")  
25  
26  # Normalize pixel values (0-255) to (0-1)  
27  X = X / 255.0  
28  
29  # Split data  
30  X_train, X_test, y_train, y_test = train_test_split(  
31      X, y, test_size=0.2, random_state=42, stratify=y  
32  )  
33  
34  print(f"Training set: {X_train.shape}")  
35  print(f"Test set: {X_test.shape}")
```

DATASET: MNIST HANDWRITTEN DIGITS

```
37 # Visualize some samples
38 fig, axes = plt.subplots(2, 5, figsize=(12, 5))
39 for i, ax in enumerate(axes.flat):
40     ax.imshow(X_train[i].reshape(28, 28), cmap='gray')
41     ax.set_title(f'Label: {y_train[i]}')
42     ax.axis('off')
43 plt.tight_layout()
44 plt.show()
```


DATASET: MNIST HANDWRITTEN DIGITS

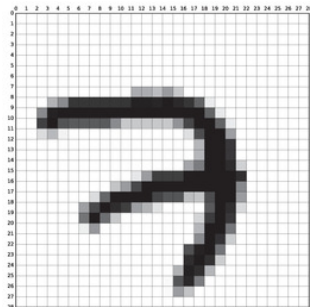
MNIST Dataset Characteristics:

- **Samples:** 70,000 handwritten digits
- **Features:** 784 (28x28 pixels)
- **Classes:** 10 digits (0-9)
- **Task:** Multi-class classification

Why MNIST untuk Neural Networks?

- Classic benchmark dataset
- Moderate complexity
- Good untuk demonstrating NN capabilities
- Well-studied dan understood

DATASET: MNIST HANDWRITTEN DIGITS⁵



(a) MNIST sample belonging to the digit '7'.



(b) 100 samples from the MNIST training set.

DATASET: MNIST HANDWRITTEN DIGITS

Data Preprocessing:

- Normalize pixel values ($0-255 \rightarrow 0-1$)
- No feature scaling needed (already same scale)
- Stratified sampling untuk maintain class distribution

Challenge:

- High-dimensional input space
- Non-linear decision boundaries
- Perfect untuk neural networks!

MLPCLASSIFIER IMPLEMENTATION

```
1 from sklearn.neural_network import MLPClassifier
2 from sklearn.metrics import accuracy_score,
   classification_report, confusion_matrix
3 import seaborn as sns
4 import time
5
6 # MLP Classifier dengan single hidden layer
7 mlp_simple = MLPClassifier(
8     hidden_layer_sizes=(100,),
9     activation='relu',
10    solver='adam',
11    alpha=0.001,
12    batch_size=256,
13    learning_rate='constant',
14    learning_rate_init=0.001,
15    max_iter=100,
16    random_state=42,
17    early_stopping=True,
18    validation_fraction=0.1,
19    n_iter_no_change=10
20 )
```

MLPCLASSIFIER IMPLEMENTATION

```
21 print("Training MLP classifier...")
22 start_time = time.time()
23 mlp_simple.fit(X_train, y_train)
24 training_time = time.time() - start_time
25
26 print(f"Training completed in {training_time:.2f}
      seconds")
27 print(f"Number of iterations: {mlp_simple.n_iter_}")
28
29 # Predictions
30 y_pred = mlp_simple.predict(X_test)
31 y_pred_proba = mlp_simple.predict_proba(X_test)
32
33 # Evaluation
34 accuracy = accuracy_score(y_test, y_pred)
35 print(f"Test Accuracy: {accuracy:.4f}")
36
37 print("\nClassification Report:")
38 print(classification_report(y_test, y_pred))
```

MLPCLASSIFIER IMPLEMENTATION

```
39 # Confusion Matrix
40 cm = confusion_matrix(y_test, y_pred)
41 plt.figure(figsize=(10, 8))
42 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
43 plt.title('Confusion Matrix - MLP Classifier')
44 plt.xlabel('Predicted')
45 plt.ylabel('Actual')
46 plt.show()
```

MLPCLASSIFIER IMPLEMENTATION

MLPClassifier Key Parameters:

ARCHITECTURE

- `hidden_layer_sizes`: Tuple of layer sizes
- `activation`: 'relu', 'tanh', 'logistic'
- `solver`: 'adam', 'sgd', 'lbfgs'

TRAINING

- `alpha`: L2 regularization
- `batch_size`: Mini-batch size
- `learning_rate`: 'constant', 'adaptive'
- `max_iter`: Maximum iterations

MLPCLASSIFIER IMPLEMENTATION

MLPClassifier Key Parameters:

EARLY STOPPING

- `early_stopping`: Stop jika validation score tidak improve
- `validation_fraction`: Fraction of training data untuk validation
- `n_iter_no_change`: Stop setelah N iterations tanpa improvement

Performance Expectations:

- Simple MLP: 90-95% accuracy
- Well-tuned MLP: 95-98% accuracy
- Deep CNN: >99% accuracy

MLPClassifier IMPLEMENTATION

MLPClassifier Key Parameters: Computational Considerations:

- Training time: Beberapa menit untuk 10,000 samples
- Memory usage: Moderate
- Suitable untuk CPU training

```
1 # Check model architecture
2 print(f"Number of layers: {mlp_simple.n_layers}")
3 print(f"Number of outputs: {mlp_simple.n_outputs}")
4 print(f"Output activation: {mlp_simple.out_activation}")
```

TRAINING PROCESS VISUALIZATION

```
1  # Plot training loss curve
2  plt.figure(figsize=(12, 4))
3
4  plt.subplot(1, 2, 1)
5  plt.plot(mlp_simple.loss_curve_)
6  plt.title('Training Loss Curve')
7  plt.xlabel('Iterations')
8  plt.ylabel('Loss')
9  plt.grid(True)
10
11 # Visualize misclassified examples
12 misclassified_idx = np.where(y_pred != y_test)[0]
13 plt.subplot(1, 2, 2)
```

TRAINING PROCESS VISUALIZATION

```
14 if len(misclassified_idx) > 0:
15     # Plot first 5 misclassified examples
16     fig, axes = plt.subplots(1, 5, figsize=(15, 3))
17     for i, idx in enumerate(misclassified_idx[:5]):
18         axes[i].imshow(X_test[idx].reshape(28, 28),
19                        cmap='gray')
19         axes[i].set_title(f'True: {y_test[idx]}, Pred:
20                           {y_pred[idx]}')
21         axes[i].axis('off')
22     plt.tight_layout()
23     plt.show()
```

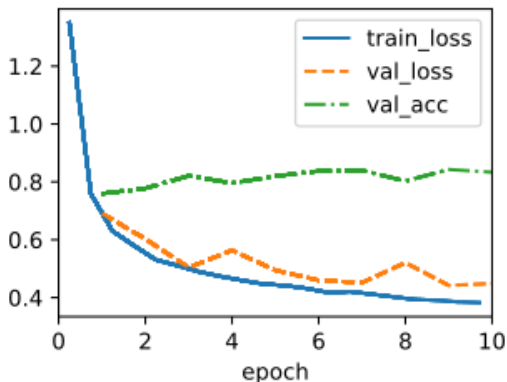
TRAINING PROCESS VISUALIZATION

```
23 # Analyze performance by class
24 class_accuracy = []
25 for digit in range(10):
26     mask = y_test == digit
27     if np.sum(mask) > 0:
28         acc = accuracy_score(y_test[mask], y_pred[mask
29                               ])
30         class_accuracy.append(acc)
31     else:
32         class_accuracy.append(0)
```

TRAINING PROCESS VISUALIZATION

```
32 plt.figure(figsize=(10, 5))
33 plt.bar(range(10), class_accuracy)
34 plt.title('Accuracy by Digit Class')
35 plt.xlabel('Digit')
36 plt.ylabel('Accuracy')
37 plt.xticks(range(10))
38 plt.grid(True, alpha=0.3)
39 plt.show()
40
41 print("Accuracy by class:")
42 for digit, acc in enumerate(class_accuracy):
43     print(f"Digit {digit}: {acc:.3f}")
```

TRAINING PROCESS VISUALIZATION⁶



⁶https://d2l.ai/chapter_multilayer-perceptrons/mlp-implementation.html

TRAINING PROCESS VISUALIZATION

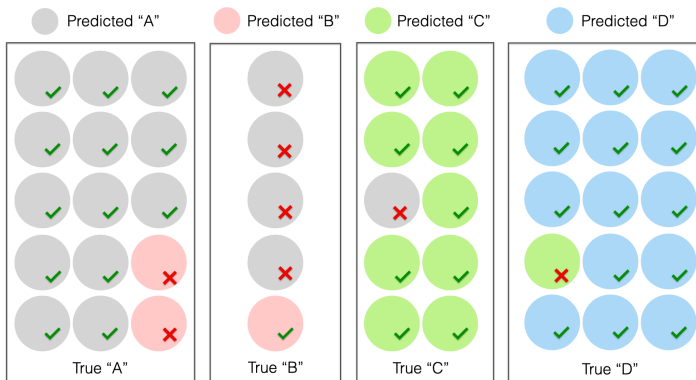
Loss Curve Interpretation:

- **Decreasing loss:** Model learning
- **Plateau:** Convergence reached
- **Oscillations:** Learning rate mungkin terlalu tinggi
- **Overfitting:** Training loss decrease, validation loss increase

Misclassification Analysis:

- Common confusions: 3-8, 4-9, 5-6
- Ambiguous handwriting
- Noisy or poorly written digits

TRAINING PROCESS VISUALIZATION⁷



⁷<https://www.evidentlyai.com/classification-metrics/multi-class-metrics>

TRAINING PROCESS VISUALIZATION

Class-wise Performance:

- Some digits lebih mudah dikenali
- Digits dengan unique shapes (0, 1) biasanya lebih akurat
- Similar-looking digits (3-8, 5-6) lebih challenging

Debugging Tips:

- Monitor loss curve selama training
- Analyze misclassified examples
- Check class-wise performance
- Tune hyperparameters berdasarkan insights

ARCHITECTURE COMPARISON: DIFFERENT NETWORK DESIGNS

```
1  # Compare different architectures
2  architectures = {
3      'Small (50)': (50,),
4      'Medium (100, 50)': (100, 50),
5      'Large (200, 100, 50)': (200, 100, 50),
6      'Wide (300)': (300,),
7      'Deep (100, 100, 100)': (100, 100, 100)
8  }
9
10 results = []
```

ARCHITECTURE COMPARISON: DIFFERENT NETWORK DESIGNS

```
11 for name, hidden_layers in architectures.items():
12     print(f"\nTraining {name} architecture...")
13
14     mlp = MLPClassifier(
15         hidden_layer_sizes=hidden_layers,
16         activation='relu',
17         solver='adam',
18         alpha=0.001,
19         batch_size=256,
20         max_iter=100,
21         random_state=42,
22         early_stopping=True,
23         n_iter_no_change=10
24     )
25
26     start_time = time.time()
27     mlp.fit(X_train, y_train)
28     training_time = time.time() - start_time
```

ARCHITECTURE COMPARISON: DIFFERENT NETWORK DESIGNS

```
29     y_pred = mlp.predict(X_test)
30     accuracy = accuracy_score(y_test, y_pred)
31
32     results.append({
33         'Architecture': name,
34         'Hidden Layers': hidden_layers,
35         'Accuracy': accuracy,
36         'Training Time (s)': training_time,
37         'Iterations': mlp.n_iter_,
38         'Parameters': sum(layer.size for layer in mlp.
39                             coefs_)
40     })
41
42 results_df = pd.DataFrame(results)
43 print("\nArchitecture Comparison:")
44 print(results_df.round(4))
```

ARCHITECTURE COMPARISON: DIFFERENT NETWORK DESIGNS

```
44 # Visualize comparison
45 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
46
47 # Accuracy comparison
48 ax1.bar(results_df['Architecture'], results_df['
    Accuracy'])
49 ax1.set_title('Accuracy by Architecture')
50 ax1.set_ylabel('Accuracy')
51 ax1.tick_params(axis='x', rotation=45)
52 ax1.grid(True, alpha=0.3)
53
54 # Training time comparison
55 ax2.bar(results_df['Architecture'], results_df['
    Training Time (s)'])
56 ax2.set_title('Training Time by Architecture')
57 ax2.set_ylabel('Time (seconds)')
58 ax2.tick_params(axis='x', rotation=45)
59 ax2.grid(True, alpha=0.3)
```

ARCHITECTURE COMPARISON: DIFFERENT NETWORK DESIGNS

```
60 plt.tight_layout()
61 plt.show()
62
63 # Parameters vs Accuracy
64 plt.figure(figsize=(10, 6))
65 plt.scatter(results_df['Parameters'], results_df['
    Accuracy'], s=100)
66 for i, row in results_df.iterrows():
67     plt.annotate(row['Architecture'],
68                 (row['Parameters'], row['Accuracy']),
69                 xytext=(5, 5), textcoords='offset
                    points')
70 plt.xlabel('Number of Parameters')
71 plt.ylabel('Accuracy')
72 plt.title('Parameters vs Accuracy')
73 plt.grid(True, alpha=0.3)
74 plt.show()
```

ARCHITECTURE COMPARISON: DIFFERENT NETWORK DESIGNS

```
75 # Best architecture
76 best_idx = results_df['Accuracy'].idxmax()
77 best_arch = results_df.loc[best_idx]
78 print(f"\nBest architecture: {best_arch['Architecture']}]}")
79 print(f"Best accuracy: {best_arch['Accuracy']:.4f}")
80 print(f"Training time: {best_arch['Training Time (s)']:.2f}s")
```

HYPERPARAMETER TUNING UNTUK MLP

```
1 from sklearn.model_selection import GridSearchCV
2 from sklearn.neural_network import MLPClassifier
3 import warnings
4 warnings.filterwarnings('ignore')
5
6 # Define parameter grid
7 param_grid = {
8     'hidden_layer_sizes': [(50,), (100,), (50, 50),
9                             (100, 50)],
10    'activation': ['relu', 'tanh'],
11    'alpha': [0.0001, 0.001, 0.01],
12    'learning_rate_init': [0.001, 0.01],
13    'batch_size': [128, 256]
14 }
```


HYPERPARAMETER TUNING UNTUK MLP

```
14 # Base MLP classifier
15 mlp_base = MLPClassifier(
16     solver='adam',
17     max_iter=50, # Smaller untuk faster tuning
18     random_state=42,
19     early_stopping=True,
20     n_iter_no_change=5
21 )
22
23 # Grid Search dengan cross-validation
24 print("Starting Grid Search...")
25 grid_search = GridSearchCV(
26     mlp_base,
27     param_grid,
28     cv=3, # 3-fold cross-validation
29     scoring='accuracy',
30     n_jobs=-1, # Use all available cores
31     verbose=1
32 )
```

HYPERPARAMETER TUNING UNTUK MLP

```
33 # Fit grid search (mungkin memakan waktu)
34 grid_search.fit(X_train, y_train)
35
36 print("\nGrid Search completed!")
37 print(f"Best parameters: {grid_search.best_params}")
38 print(f"Best cross-validation score: {grid_search.
    best_score_:.4f}")
39
40 # Evaluate best model
41 best_mlp = grid_search.best_estimator_
42
43 # Retrain dengan lebih banyak iterations
44 best_mlp.set_params(max_iter=200, early_stopping=True)
45 best_mlp.fit(X_train, y_train)
46
47 # Final evaluation
48 y_pred_best = best_mlp.predict(X_test)
49 accuracy_best = accuracy_score(y_test, y_pred_best)
50
51 print(f"Best model test accuracy: {accuracy_best:.4f}"
    )
```

HYPERPARAMETER TUNING UNTUK MLP

```
53 # Compare dengan default model
54 default_mlp = MLPClassifier(random_state=42)
55 default_mlp.fit(X_train, y_train)
56 y_pred_default = default_mlp.predict(X_test)
57 accuracy_default = accuracy_score(y_test,
    y_pred_default)
58
59 print(f"Default model test accuracy: {accuracy_default
    :.4f}")
60 print(f"Improvement: {accuracy_best - accuracy_default
    :.4f}")
61
62 # Visualize grid search results
63 results_df = pd.DataFrame(grid_search.cv_results_)
64
65 # Plot effect of hidden layer sizes
66 layer_results = results_df[results_df['
    param_activation'] == 'relu']
67 layer_results = layer_results[layer_results['
    param_alpha'] == 0.001]
```

HYPERPARAMETER TUNING UNTUK MLP

```
68 plt.figure(figsize=(10, 6))
69 for layers in param_grid['hidden_layer_sizes']:
70     mask = layer_results['param_hidden_layer_sizes']
71         == layers
72     if mask.any():
73         plt.plot(layer_results[mask]['
74             param_learning_rate_init'],
75                 layer_results[mask]['mean_test_score'
76                     ],
77                 'o-', label=str(layers))
78
79 plt.xlabel('Learning Rate')
80 plt.ylabel('CV Accuracy')
81 plt.title('Effect of Architecture and Learning Rate')
82 plt.legend(title='Hidden Layers')
```

HYPERPARAMETER TUNING UNTUK MLP

```
82 print("\nKey Insights from Tuning:")
83 print("- Hidden layer size optimal range: 50-100
      neurons")
84 print("- ReLU generally outperforms tanh")
85 print("- Learning rate 0.001 works well")
86 print("- Moderate regularization (alpha=0.001) is
      beneficial")
```

AGENDA

KONSEP NEURAL NETWORKS

IMPLEMENTASI DENGAN SCIKIT-LEARN

MLP UNTUK REGRESSION

LIVE CODING

PENUTUP

MLPREGRESSOR UNTUK REGRESSION PROBLEMS

```
1 from sklearn.neural_network import MLPRegressor
2 from sklearn.datasets import fetch_california_housing
3 from sklearn.metrics import mean_squared_error,
   r2_score
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.model_selection import train_test_split
6
7 # Load regression dataset
8 california = fetch_california_housing()
9 X_reg = california.data
10 y_reg = california.target
11
12 print("California Housing Dataset:")
13 print(f"Features: {X_reg.shape[1]}")
14 print(f"Samples: {X_reg.shape[0]}")
15 print(f"Target range: {y_reg.min():.2f} - {y_reg.max():.2f}")
```

MLPREGRESSOR UNTUK REGRESSION PROBLEMS

```
16  # Preprocessing untuk regression
17  scaler_X = StandardScaler()
18  scaler_y = StandardScaler()
19
20  X_reg_scaled = scaler_X.fit_transform(X_reg)
21  y_reg_scaled = scaler_y.fit_transform(y_reg.reshape
    (-1, 1)).ravel()
22
23  # Split data
24  X_train_reg, X_test_reg, y_train_reg, y_test_reg =
    train_test_split(
25      X_reg_scaled, y_reg_scaled, test_size=0.2,
        random_state=42
26  )
```


MLPREGRESSOR UNTUK REGRESSION PROBLEMS

```
27 # MLP Regressor
28 mlp_regressor = MLPRegressor(
29     hidden_layer_sizes=(100, 50),    # Two hidden
        layers
30     activation='relu',
31     solver='adam',
32     alpha=0.001,                      # Regularization
33     batch_size=256,
34     learning_rate_init=0.001,
35     max_iter=500,                     # More iterations
        untuk regression
36     random_state=42,
37     early_stopping=True
38 )
39
40 print("\nTraining MLP Regressor...")
41 mlp_regressor.fit(X_train_reg, y_train_reg)
```

MLPREGRESSOR UNTUK REGRESSION PROBLEMS

```
42 # Predictions
43 y_pred_reg = mlp_regressor.predict(X_test_reg)
44
45 # Convert back to original scale
46 y_pred_original = scaler_y.inverse_transform(
    y_pred_reg.reshape(-1, 1)).ravel()
47 y_test_original = scaler_y.inverse_transform(
    y_test_reg.reshape(-1, 1)).ravel()
48
49 # Evaluation
50 mse = mean_squared_error(y_test_original,
    y_pred_original)
51 rmse = np.sqrt(mse)
52 r2 = r2_score(y_test_original, y_pred_original)
53
54 print(f"Regression Performance:")
55 print(f"MSE: {mse:.4f}")
56 print(f"RMSE: {rmse:.4f}")
57 print(f"R Square Score: {r2:.4f}")
```

MLPREGRESSOR UNTUK REGRESSION PROBLEMS

```
58 # Visualize predictions vs actual
59 plt.figure(figsize=(12, 5))
60
61 plt.subplot(1, 2, 1)
62 plt.scatter(y_test_original, y_pred_original, alpha
63             =0.6)
64 plt.plot([y_test_original.min(), y_test_original.max()
65           ],
66           [y_test_original.min(), y_test_original.max()
67             ], 'r--', lw=2)
68
69 plt.xlabel('Actual Prices')
70 plt.ylabel('Predicted Prices')
71 plt.title('MLP Regressor: Actual vs Predicted')
72
73 plt.subplot(1, 2, 2)
74 residuals = y_test_original - y_pred_original
75 plt.scatter(y_pred_original, residuals, alpha=0.6)
76 plt.axhline(y=0, color='r', linestyle='--')
77 plt.xlabel('Predicted Prices')
78 plt.ylabel('Residuals')
79 plt.title('Residual Plot')
```

MLPREGRESSOR UNTUK REGRESSION PROBLEMS

```
76 plt.tight_layout()
77 plt.show()
78
79 # Compare dengan linear regression
80 from sklearn.linear_model import LinearRegression
81
82 lr = LinearRegression()
83 lr.fit(X_train_reg, y_train_reg)
84 y_pred_lr = lr.predict(X_test_reg)
85 y_pred_lr_original = scaler_y.inverse_transform(
    y_pred_lr.reshape(-1, 1)).ravel()
86
87 mse_lr = mean_squared_error(y_test_original,
    y_pred_lr_original)
88 r2_lr = r2_score(y_test_original, y_pred_lr_original)
```

MLPREGRESSOR UNTUK REGRESSION PROBLEMS

```
86 print(f"\nComparison with Linear Regression:")
87 print(f"Linear Regression - MSE: {mse_lr:.4f}, R
    Square: {r2_lr:.4f}")
88 print(f"MLP Regressor - MSE: {mse:.4f}, R Square: {r2
    :.4f}")
89 print(f"Improvement: {r2 - r2_lr:.4f}")
```

AGENDA

KONSEP NEURAL NETWORKS

IMPLEMENTASI DENGAN SCIKIT-LEARN

MLP UNTUK REGRESSION

LIVE CODING

PENUTUP

LIVE CODING SESSION

Kita akan implementasi lengkap neural networks dengan scikit-learn:

- Load dan preprocess MNIST dataset
- Implement MLPClassifier dengan different architectures
- Visualisasi training process dan results
- Tune hyperparameters dengan GridSearchCV
- Bandingkan different activation functions
- Implement MLPRegressor untuk regression
- Analisis model performance dan limitations

MARI BANGUN NEURAL NETWORKS!

```
1  # Prepare for live coding
2  import numpy as np
3  import pandas as pd
4  import matplotlib.pyplot as plt
5  import seaborn as sns
6  from sklearn.datasets import fetch_openml
7  from sklearn.neural_network import MLPClassifier, MLPRegressor
8  from sklearn.model_selection import train_test_split, GridSearchCV
9  from sklearn.metrics import accuracy_score, classification_report,
   confusion_matrix
10 from sklearn.metrics import mean_squared_error, r2_score
11 from sklearn.preprocessing import StandardScaler
12 import time
```


AGENDA

KONSEP NEURAL NETWORKS

IMPLEMENTASI DENGAN SCIKIT-LEARN

MLP UNTUK REGRESSION

LIVE CODING

PENUTUP

RANGKUMAN PEKAN INI

- **Neural Networks:** Inspired by biological neurons
- **Perceptron:** Basic building block dengan weights dan activation function
- **Activation Functions:** ReLU, Sigmoid, Tanh, Softmax
- **MLP Architecture:** Input, hidden, dan output layers
- **MLPClassifier:** Untuk classification problems
- **MLPRegressor:** Untuk regression problems
- **Hyperparameter Tuning:** Critical untuk performance optimal
- **Limitations:** Computational cost, black box nature

RANGKUMAN HARI INI

```
1  # Neural networks vs traditional ML
2  comparison = {
3      'Neural Networks': ['Better untuk complex patterns',
4                          'Automatic feature learning',
5                          'High computational requirements'],
6      'Traditional ML': ['More interpretable',
7                          'Faster training',
8                          'Better untuk small datasets']
9  }
```

TUGAS PEKAN INI

Dataset: MNIST atau dataset classification/regression lainnya

Tugas: Buat analisis neural networks dalam Jupyter Notebook:

1. Implement MLPClassifier dengan minimal 3 different architectures
2. Bandingkan performance dan training time
3. Visualisasi training curves dan misclassifications
4. Lakukan hyperparameter tuning dengan GridSearchCV
5. Bandingkan dengan traditional ML model (Random Forest, etc.)
6. Implement MLPRegressor untuk regression problem
7. Analisis strengths dan limitations neural networks

Deadline: Sebelum pertemuan 11

TUGAS PERTEMUAN 10

```
1  # Template untuk memulai tugas
2  from sklearn.neural_network import MLPClassifier, MLPRegressor
3  from sklearn.model_selection import train_test_split, GridSearchCV
4  from sklearn.metrics import accuracy_score, classification_report
5  from sklearn.datasets import fetch_openml
6  import matplotlib.pyplot as plt
7  import numpy as np
```

```
1  # Resources untuk belajar lebih lanjut
2  resources = {
3      'Scikit-learn Neural Networks': 'https://scikit-learn.org/stable/modules/
      neural_networks_supervised.html',
4      'Neural Networks and Deep Learning Book': 'http://
      neuralnetworksanddeeplearning.com/',
5      'Interactive Neural Network Playground': 'https://playground.tensorflow.org/
      ',
6      'MNIST Database': 'http://yann.lecun.com/exdb/mnist/'
7  }
```

